



CSC 309: Artificial Intelligence

Module 5B: Expert Systems & AI Programming Languages

Instructors:
Dr. Muhammad Yusuf
Najib Abdulrazak
Bilal Lawal Alhassan
Lawan Yusuf Abdullahi

1. Introduction to Expert Systems

While AI branches like Machine Learning attempt to learn hidden patterns from vast oceans of raw data, and NLP attempts to parse human communication, **Expert Systems (ES)** take a fundamentally different, symbolic approach. They belong to the "Good Old-Fashioned AI" (GOFAI) paradigm, where human knowledge is explicitly hard-coded into the machine.

An Expert System is a computer program that uses artificial intelligence technologies to simulate the judgment, behavior, and decision-making processes of a human or an organization that has highly specialized expert knowledge in a narrowly defined domain.

Conventional Programming vs. Expert Systems

To grasp Expert Systems, you must understand how their architecture differs from traditional software development:

- **Conventional Program:** *Data + Algorithms = Output*
 - In standard software (like a payroll system in Java or C), the knowledge about how the system works and the control logic (if/else loops) are intertwined and compiled together within the source code.
 - If a tax rule changes, the programmer must rewrite the code, recompile, and redeploy the system.
- **Expert System:** *Knowledge + Inference = Advice/Decision*
 - Knowledge is kept strictly separate from the control logic (the Inference Engine).
 - The system is highly modular. If a medical guideline changes, a domain expert simply updates a rule in the Knowledge Base without ever touching the system's core programming or requiring a recompile.

Historical Milestones

Expert Systems were the first truly successful commercial applications of AI, leading to the "AI Boom" of the 1980s.

- **DENDRAL (late 1960s):** Developed at Stanford, it analyzed mass spectrometry data to determine the molecular structure of organic compounds. It proved that AI could do more than play games or prove theorems—it could do real scientific work.
- **MYCIN (1970s):** A landmark system designed to diagnose infectious blood diseases and recommend antibiotic therapies. It factored in the patient's body weight to prescribe exact dosages. In trials, MYCIN's diagnostic accuracy was around 69%, outperforming junior doctors and matching the performance of Stanford's senior infectious disease specialists.
- **XCON / R1 (1980s):** Developed for Digital Equipment Corporation (DEC). It was used to automatically configure customer computer system orders to ensure all parts were compatible. It saved DEC an estimated ^{\$40} million a year, sparking the corporate rush to adopt Expert Systems.

- **PROSPECTOR (1981):** A geological expert system that successfully predicted the location of an undiscovered Molybdenum deposit in Washington State, leading to millions of dollars in mining value.

2. Characteristics and Requirements of Expert Systems

For a system to be rigorously classified as an "Expert System," it must exhibit several distinct characteristics:

1. **High Performance:** The system must respond at a level of competence equal to or better than a human expert in that specific domain.
2. **Domain-Specific (Narrow Focus):** An expert system only knows its niche. A medical diagnostic ES knows nothing about car repair. They are the epitome of "Narrow AI."
3. **Understandability (Self-Explanation):** A human expert can explain *why* they made a decision. An ES must also have the capacity to explain the steps of its reasoning to the user. (A doctor won't amputate a leg just because a black-box AI says so; they need to see the logical proof).
4. **Reliability and Robustness:** It must not be prone to catastrophic errors, as these systems are often deployed in high-stakes environments (medicine, aviation, finance).
5. **Highly Responsive:** It must process information and provide answers in a timely manner, sometimes in real-time (e.g., automated nuclear plant monitoring).

3. Architecture and Core Components

A typical Expert System is built upon a modular architecture consisting of five distinct components:

1. The Knowledge Base (KB)

This is the heart of the system. Unlike a standard relational database that stores passive rows and columns of data, a KB stores relationships, facts, and logical rules.

- **Factual Knowledge:** Information widely accepted by experts in the domain (e.g., "Water boils at 100°C ", "Aspirin is a blood thinner").
- **Heuristic Knowledge:** Rules of thumb, experiential guesses, and intuitive strategies used by human experts. Humans rarely make decisions on absolute facts; they use heuristics (e.g., "If the patient has a high fever and a rash, suspect Measles unless proven otherwise").
- **Representation:** Typically stored as **IF-THEN production rules** (e.g., IF condition A AND condition B, THEN conclusion C).

2. The Inference Engine

This is the "brain" or the control mechanism of the system. It is a piece of software that

manipulates the Knowledge Base to deduce new information and arrive at a conclusion.

The engine must sometimes deal with **Conflict Resolution**. If two IF-THEN rules are triggered simultaneously by the same data, which one fires first?

- *Specificity*: Fire the most specific rule (the one with the most IF conditions).
- *Priority*: Fire the rule explicitly tagged by the developer as higher importance.
- *Recency*: Fire the rule that uses the most recently added facts.

The Inference Engine navigates the rules using two primary strategies:

A. Forward Chaining (Data-Driven)

Starts with the known facts and applies rules to extract new facts until a goal or conclusion is reached.

- *Example KB*:
 - **Rule 1**: IF engine_turns_over = False AND lights = Dim, THEN battery = Dead.
 - **Rule 2**: IF battery = Dead, THEN action = Replace_Battery.
- *Execution Steps*:
 1. User inputs facts: engine_turns_over = False, lights = Dim.
 2. Engine scans rules. The facts satisfy the "IF" condition of Rule 1.
 3. Rule 1 fires, adding a new fact to the KB: battery = Dead.
 4. Engine scans again. The new fact satisfies the "IF" condition of Rule 2.
 5. Rule 2 fires, outputting the final conclusion: "Replace Battery."
- *Best for*: Planning, monitoring, designing, and synthesizing systems where the final outcome is unknown.

B. Backward Chaining (Goal-Driven)

Starts with a hypothesized goal and works backward through the rules to see if there are facts in the KB (or inputs from the user) to support it.

- *Example KB*: (Same rules as above).
- *Execution Steps*:
 1. Hypothesis/Goal: The user asks, "Should I Replace_Battery?"
 2. Engine looks for rules where "Replace_Battery" is the THEN part. It finds Rule 2.
 3. To satisfy Rule 2, it needs the fact battery = Dead. This becomes the new sub-goal.
 4. Engine looks for rules where "battery = Dead" is the THEN part. It finds Rule 1.
 5. To satisfy Rule 1, it needs to know if engine_turns_over = False and lights = Dim.
 6. It asks the user: "Are your lights dim?" and "Does the engine turn over?" If the user says Yes and No, the sub-goals are proven, cascading forward to prove the ultimate hypothesis.
- *Best for*: Diagnostics, troubleshooting, and debugging where you have a suspicion of the cause and want to prove it.

3. The User Interface (UI)

The mechanism by which the end-user interacts with the system. A well-designed UI hides the complex logic and often incorporates Natural Language Processing (NLP) to allow the user to query the system using everyday language.

4. The Explanation Facility

This module traces the execution path of the inference engine.

- **The "How" Query:** Explains how the system reached a conclusion. (e.g., "I concluded you have Malaria *because* you had a fever AND traveled to an endemic zone").
- **The "Why" Query:** Explains why the system is asking the user a specific question. (e.g., User asks "Why do you need to know if I have a headache?". System replies: "Because I am trying to prove the sub-goal 'Patient has Migraine' via Rule 42").

5. Knowledge Acquisition Facility

A specialized interface that allows developers or domain experts to easily add, modify, or delete rules and facts in the Knowledge Base without needing to understand the underlying LISP or Prolog code.

4. The Development Lifecycle (Knowledge Engineering)

Building an Expert System is fundamentally different from standard software engineering. The specialized process is called **Knowledge Engineering**. The key personnel include:

- **Domain Expert:** The human with the specialized knowledge (e.g., a senior cardiologist). They do not need to know anything about computers.
- **Knowledge Engineer:** The AI specialist who extracts the knowledge from the expert, structures it into logic, and writes the rules.
- **End User:** The person who will ultimately use the system for advice (e.g., a junior medical student).

Phases of Knowledge Engineering:

1. **Problem Identification:** Determining if the problem actually requires an ES. (Is it narrow enough? Does it require symbolic reasoning rather than math? Are human experts available and willing to help?).
2. **Knowledge Acquisition (The "Knowledge Acquisition Bottleneck"):** This is the hardest part of building an ES. The Knowledge Engineer interviews the Domain Expert. Human experts often rely on "gut feeling" or subconscious pattern recognition and struggle to articulate their actual step-by-step reasoning. Techniques include structured interviews, protocol analysis (observing the expert at work), and role-playing.
3. **Knowledge Representation:** Formalizing the extracted, messy human knowledge into strict IF-THEN rules, Predicate Logic, or Semantic Frames.
4. **Implementation:** Building the Knowledge Base and connecting it to the Inference Engine

using an AI language or ES Shell (like CLIPS).

5. **Testing and Validation:** Running the system against historical case studies (e.g., old medical records) to ensure its output matches what the human expert *actually* decided in the past.
6. **Maintenance:** Knowledge decays. Medical guidelines change; tax laws update. A critical phase is continuously pruning and updating the rulebase so the system doesn't give obsolete advice.

5. Limitations and Pitfalls of Expert Systems

While powerful, the classical Expert System approach (GOF AI) has significant drawbacks that led to the decline of their dominance in favor of modern Machine Learning:

1. **The Brittleness Problem:** ES are extremely rigid. If a problem falls even slightly outside their explicitly programmed rules, they fail completely and often absurdly. They cannot make "educated guesses" outside their domain.
2. **Lack of Common Sense:** An ES diagnosing car engines doesn't actually know what a car *is*, or that it has physical mass, or that a user cannot fit a car in their pocket. It just manipulates text symbols. This lack of generalized worldly common sense leads to bizarre errors.
3. **Inability to Learn:** A classical ES does not learn from its mistakes. If it gives a wrong diagnosis 100 times, it will give it a 101st time unless a human Knowledge Engineer manually intervenes and alters the rule.
4. **The Knowledge Acquisition Bottleneck:** Extracting rules from human experts is incredibly slow, expensive, and tedious. It proved impossible to write enough rules to encompass complex, ambiguous environments (like driving a car).

6. Programming Languages for AI and Expert Systems

Historically, AI development relied on highly specialized programming languages designed for symbolic logic and tree manipulation rather than numerical calculation (like Fortran or C).

1. LISP (LISt Processing)

- Created by John McCarthy in 1958 (the second oldest high-level language still in use, after Fortran).
- **Paradigm:** Functional programming.
- **Features:** It heavily utilizes parentheses. Its most powerful feature is *homoiconicity*—it treats its own code as data. This meant LISP programs could easily write, modify, and execute new LISP programs on the fly, making it highly favored for early AI research in the United States.

2. Prolog (PROgramming in LOGic)

- Created by Alain Colmerauer in 1972.
- **Paradigm:** Declarative programming.

- **Features:** In Java or C, you tell the computer *how* to do something step-by-step. In Prolog, you simply tell the computer *what is true* (facts) and *what the rules are*. Prolog has a **built-in Inference Engine using Backward Chaining**.
- **Example snippet:**
 - `parent(john, mary).` (Fact: John is the parent of Mary)
 - `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).` (Rule: X is the grandparent of Y IF X is the parent of Z AND Z is the parent of Y).
- **Status:** The classical standard for building Expert Systems and natural language parsers, particularly favored in Europe and Japan.

3. Modern AI Languages & Business Rules Engines

While LISP and Prolog are mostly relegated to academia today, the *concept* of Expert Systems is alive and well in the corporate world, though they are now usually referred to as **Business Rules Management Systems (BRMS)**.

- **Python:** The undisputed king of modern AI. While primarily used for Machine Learning, its readability makes it easy to script modern rule-based engines.
- **Java (Drools):** In enterprise environments, systems like JBoss Drools are used to build massive, scalable Expert Systems for calculating bank loans, insurance premiums, and fraud detection.
- **C++:** Used when the Inference Engine requires hyper-optimized, real-time performance, such as rule-based triggers in high-frequency algorithmic trading.

Summary: Module 5B demonstrates how human expertise is captured, structured, and automated. By strictly separating the Knowledge Base from the Inference Engine, Expert Systems allow for flexible, explainable AI capable of sophisticated diagnostic reasoning. While classical languages like LISP and Prolog pioneered this field, and classical ES face limitations in learning and common sense, the principles of Knowledge Engineering remain highly relevant in modern enterprise systems design.